

A Neural Network for Authorship Stylometry

Given a passage written by one of several known authors — or by nobody we know — can we recover who wrote it from writing style alone? Where K-means asked whether style clusters, here we ask a supervised question: train a neural network on labelled passages and let it learn the decision boundaries between authors directly, including a boundary for “none of the above”.

1 Classifying Authors with an NN

The task. A five-author classifier assumes every passage was written by one of the known authors. In the real world that assumption fails: a passage may come from an *unknown* author, and a five-author model will confidently mislabel it as whichever known author it resembles most. We therefore treat it as a plain **six-class** classification problem: the five known LessWrong authors plus an explicit sixth class, `none_of_the_5_authors`, that the network must learn to recognise.

Dataset and preprocessing. The five known authors are `Eliezer Yudkowsky`, `John Wentworth`, `Raemon`, `Scott Alexander`, and `Zvi`, with roughly 150 passages each (723 passages in total). The “none” class is built from **15 other LessWrong authors** not among the five (`ricraz`, `benquo`, `abramdemski`, `sarahconstantin`, `holdenkarnofsky`, `gordon-seidoh-worley`, `screwtape`, `buck`, `turntrout`, `nunosempere`, `benito`, `petermccluskey`, `joe-carlsmith`, `adamshimi`, `tsvibt`), sampling 10 articles from each for **150 “none” passages**. Merged, this gives **873 passages over 6 classes**.

From each passage we extract **107 stylometric features** spanning seven groups: word/sentence-length statistics, vocabulary richness (type-token ratio, hapax ratio, Yule K , Simpson D , Brunét W , Honoré R), function-word frequencies, stylistic word-category rates, punctuation rates, coarse POS-tag distributions, and character-level ratios. None of these depend on *what* a passage is about, only on *how* it is written. Every feature is z-scored with `sklearn`’s `StandardScaler` (mean 0, standard deviation 1) before training. Crucially, the scaler is fit on the training split only and then applied to the validation and test splits, so no test-set statistics leak into training.

The classifier. A neural network (NN) maps a feature vector $\mathbf{x} \in \mathbb{R}^{107}$ to a probability over the $K = 6$ classes. Each hidden layer applies an affine transform followed by the **ReLU** nonlinearity,

$$\mathbf{h}^{(\ell)} = \text{ReLU}(W^{(\ell)}\mathbf{h}^{(\ell-1)} + \mathbf{b}^{(\ell)}), \quad \text{ReLU}(z) = \max(0, z),$$

with $\mathbf{h}^{(0)} = \mathbf{x}$. The final layer produces K logits that a **softmax** turns into class probabilities,

$$p_k(\mathbf{x}) = \frac{\exp(z_k)}{\sum_{j=1}^K \exp(z_j)},$$

and training minimises the multi-class **cross-entropy** loss over the training passages,

$$L = -\frac{1}{N} \sum_{i=1}^N \sum_{k=1}^K \mathbf{1}[y_i = k] \log p_k(\mathbf{x}_i).$$

Weights are learned by backpropagation with the **Adam** optimiser (`sklearn.neural_network.MLPClassifier`, `solver="adam"`). All hidden layers use 64 units; we vary only the *number* of such layers.

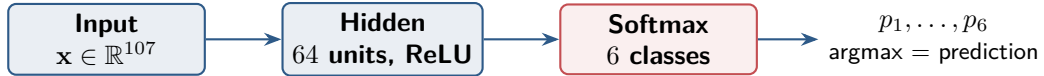


Figure 1: The winning architecture: a single hidden layer of 64 ReLU units feeding a 6-way softmax (five known authors plus “none”). Deeper variants stack more 64-unit layers between input and output (Section 2).

Training protocol. We split the 873 passages **60/20/20** (train / validation / test), stratified by class, and use the validation split to choose a model before ever touching the test split. Specifically we grid-search over three feature subsets (the top 30, top 50, and all 107 features) crossed with four network depths, training each combination on the train split and scoring it on validation. The winner is then **retrained on train+validation combined** and evaluated *once* on the held-out test set.

Which features? The top-30 and top-50 subsets are the most informative features as ranked by **Random Forest permutation importance**: we fit a random forest (`n_estimators=500` , `max_depth=5`) and measure, for each feature, how much test accuracy drops when that feature’s values are randomly shuffled. Permutation importance is multivariate and redundancy-aware — when several stylometric features carry overlapping signal, it credits the one the forest actually relies on — so it is a sharper subset selector than scoring each feature in isolation. As it turns out the distinction barely matters here: the full 107-feature model wins on validation regardless, so the subsets serve only to confirm that no small hand-picked group beats simply using everything.

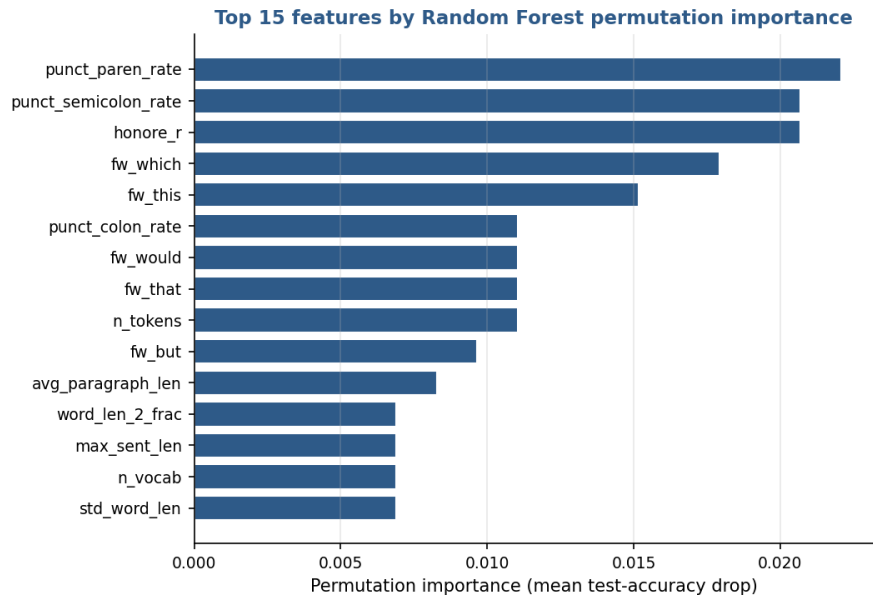


Figure 2: The 15 most informative features by Random Forest permutation importance (mean drop in held-out accuracy when a feature is shuffled). Punctuation habits (parenthesis, semicolon, colon rates), a vocabulary-richness measure (Honoré’s R), and a handful of function words (`which` , `this` , `would` , `that`) dominate — exactly the topic-independent, stylistic signals we want.

With only ~ 520 training passages and thousands of weights, an NN can memorise the training set. We pass `early_stopping=True` : `sklearn` holds out 10% of the training data as an internal validation set

and halts when its score fails to improve for `n_iter_no_change=15` consecutive epochs (`batch_size=32` , `max_iter=500`). This curbs over-fitting and makes the depth comparison fair — every architecture is trained under the same stopping rule.

Model selection on the validation set. Figure 3 shows validation accuracy for every feature-subset $\times$ depth combination. The clear winner is **all 107 features with a single hidden layer** (validation accuracy 0.943). Two patterns already stand out: using all 107 features never hurts, and the deepest network (50 layers) collapses to chance — a phenomenon we return to in Section 2.

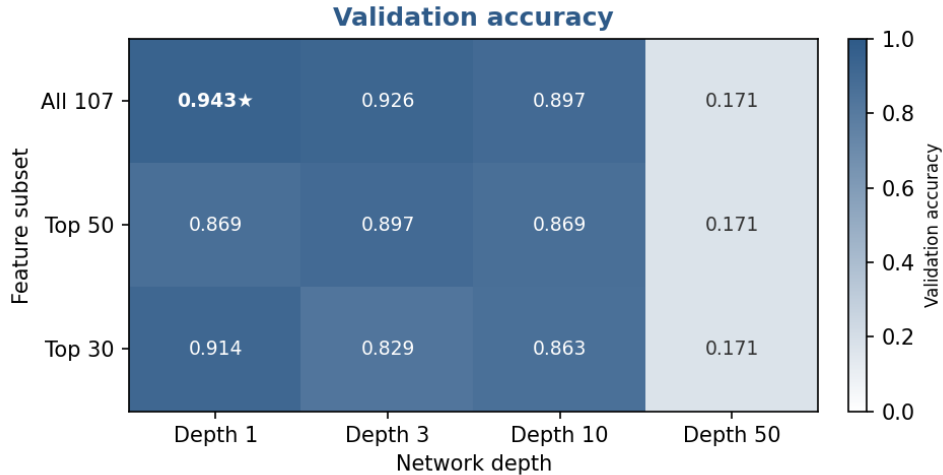


Figure 3: Validation accuracy across feature subsets (rows) and network depths (columns). The starred cell — all 107 features, depth 1 — is the selected model. Depth 50 sits at 0.171, barely above the six-class random baseline of $1/6 \approx 0.167$.

Results. Retrained on train+validation (698 passages) and evaluated on the 175 test passages, the selected model attains the scores in Table 1.

Table 1: Test-set performance of the selected NN (all 107 features, depth 1).

Metric	Value
Accuracy	0.949
Weighted F1	0.949

The confusion matrix (Figure 4) shows a strong diagonal. The striking result is the `none_of_the_5_authors` class itself: **precision, recall, and F1 are all 1.00**. Not a single unknown-author passage is misattributed to one of the five, and not a single known-author passage is dumped into the “none” bin — the 15 unknown authors, pooled together, occupy a region of style-space cleanly separable from the five targets. Among the five known authors the few errors concentrate on `Raemon` (recall 0.88) and `Scott Alexander` (recall 0.90), whose passages are occasionally confused with one another or with `Yudkowsky` ; `Zvi` is recovered almost perfectly.

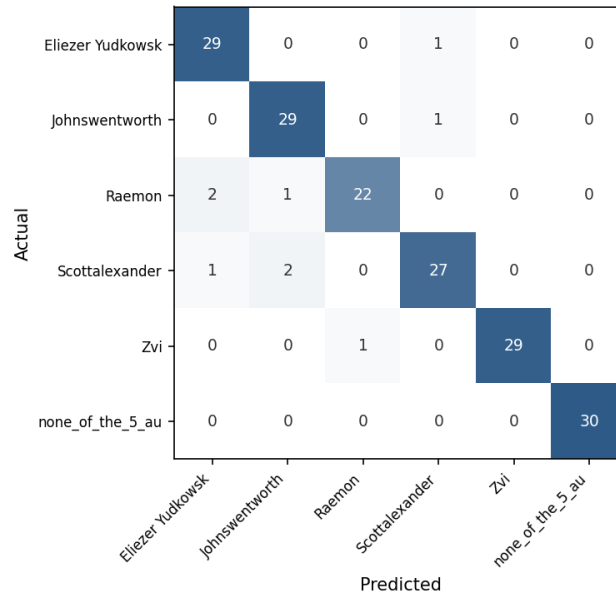


Figure 4: Test-set confusion matrix (rows = actual, columns = predicted). The `none` row and column are perfectly isolated.

Why add a class instead of simply flagging low-confidence predictions from a five-author model as “unknown”? We tried that indirect route, thresholding the five-class softmax probabilities, and it failed completely: unknown passages routinely produced *high*-confidence predictions for one of the five authors, so a confidence threshold caught essentially none of them. Giving the network labelled “none” examples and letting it learn the boundary directly is far more effective — here, perfectly so. The contrast against 15 stylistically diverse “other” authors also appears to *sharpen* the boundaries among the five targets: the model learns not just what each target author looks like, but what they do *not* look like.

2 How Deep Should the Network Be?

A natural instinct is that deeper networks are more powerful, so they should classify better. Our grid search says otherwise. Figure 5 plots validation accuracy against depth for each feature subset.

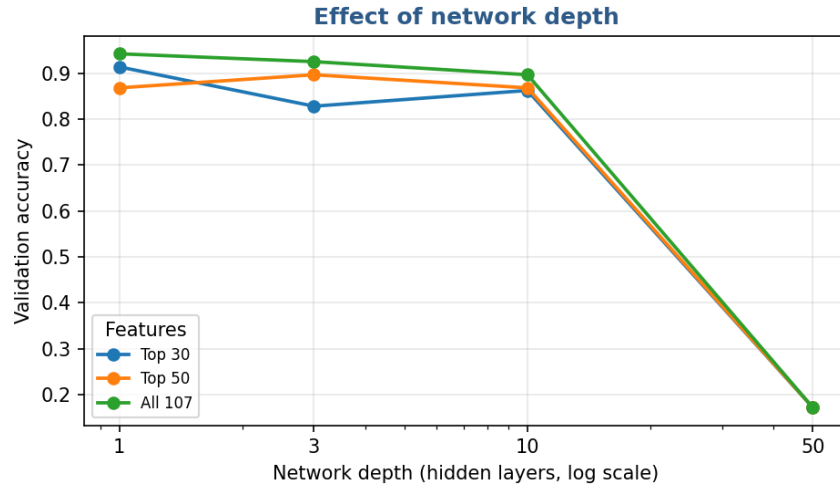


Figure 5: Validation accuracy vs. network depth (number of 64-unit hidden layers, log scale). Performance is flat-to-slightly-declining from depth 1 to depth 10, then **collapses to chance at depth 50**.

From depth 1 to depth 10 accuracy barely moves — the extra layers buy nothing because the decision boundaries between these classes are already well within reach of a shallow network on 107 informative features. At depth 50 every configuration collapses: validation accuracy falls to 0.171, essentially the random baseline of $1/6 \approx 0.167$. The network has stopped learning anything.

This is not over-fitting — an over-fit network would still score well above chance on validation. It is an *optimisation* failure. Stacking 50 plain fully-connected ReLU layers with no residual connections or normalisation produces vanishing gradients: the error signal is attenuated layer by layer during backpropagation, so the early layers receive almost no useful gradient. Combined with early stopping on a small dataset, training halts before the network escapes its near-random initialisation. The lesson is concrete: for tabular stylometric features, **a shallow NN is not just sufficient but preferable**.

Are the gradients actually vanishing? The vanishing-gradient explanation is testable, so we test it. We re-implement the same architecture in numpy — Glorot-uniform initialisation with bound $\sqrt{6/(\text{fan_in} + \text{fan_out})}$ (exactly what `sklearn` uses for ReLU layers), ReLU hidden units, softmax output, cross-entropy loss — and read off the gradient of the loss with respect to every layer’s weights from a single forward/backward pass on the standardised features. Averaging over 50 random initialisations, Figure 6 reports the root-mean-square (RMS) of the weight gradients at each layer, at the moment training begins.

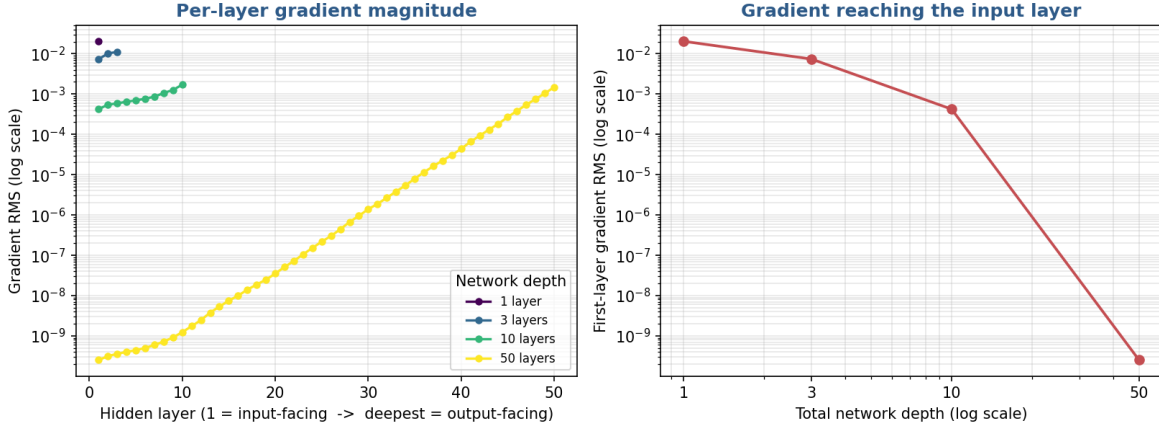


Figure 6: Gradient health at initialisation. *Left:* RMS weight gradient per layer, from the input-facing layer (1) to the output-facing layer, one curve per depth (log scale). *Right:* the gradient reaching the input-facing first layer as a function of total depth. Both axes are logarithmic.

The signature is unmistakable. In the shallow networks the gradient is roughly uniform across layers — at depth 1 the input layer already sees an RMS gradient of 2.0×10^{-2} , only $3.5\times$ smaller than the output layer. As depth grows the gradient decays geometrically on its way back to the input, so the first layer is starved (Table 2). By 50 layers the input-facing layer receives an RMS gradient of 2.5×10^{-10} — eight orders of magnitude weaker than in the shallow net, and 2×10^7 times weaker than the output layer of the *same* network.

Table 2: RMS weight gradient at initialisation, averaged over 50 inits. The first layer (input-facing) is the one that must learn feature combinations; its gradient collapses with depth.

Depth	First-layer grad	Output grad	Ratio (out / first)
1	2.0×10^{-2}	7.0×10^{-2}	3.5
3	7.4×10^{-3}	3.1×10^{-2}	4.2
10	4.2×10^{-4}	5.8×10^{-3}	13.7
50	2.5×10^{-10}	5.5×10^{-3}	2.2×10^7

A weight that sees a gradient of 10^{-10} cannot move: with early stopping monitoring a validation score that never improves, training halts within a few epochs and the deep network never leaves its random initialisation — which is exactly the chance-level accuracy we saw at depth 50. This confirms the collapse is an *optimisation* pathology, not a capacity or over-fitting one. The standard fixes (residual connections, batch/layer normalisation, He initialisation) all exist precisely to keep this gradient alive; none are warranted here, because the shallow network already solves the task.

3 Conclusion

A deliberately simple model — a single hidden layer of 64 ReLU units on 107 z-scored stylometric features — attributes LessWrong passages to one of five authors or to a “none of the above” class with 95% accuracy, recognising the unknown-author class *perfectly*. Depth does not help and, past a point, destroys the model outright. The supervised picture is sharper than the unsupervised one: where K-means recovered ten-author structure at $\text{ARI} \approx 0.55$, a labelled NN separates a handful of authors almost cleanly, confirming that the stylistic signal is strong and largely linearly-accessible once the right features are in hand.

4 Setup and Reproducibility

All experiments run on CPU in well under a minute each. The module lives in `src/neural_network/`, organised into `data/` (the feature CSV and the `cleaned_5` / `cleaned_35` article sets), `scripts/` (all Python), and `outputs/` (every figure plus `results.md`). The report figures are produced by `make_report_figures.py` and `gradient_diagnostics.py`.

Dependencies. Python ≥ 3.10 with a handful of scientific packages:

- `numpy`, `pandas` — data handling
- `scikit-learn` — `MLPClassifier`, splitting, scaling, metrics
- `matplotlib` — all figures
- `nltk` — tokenisation / POS-tagging when building the “none” class from raw articles (a regex fallback is used if NLTK data is unavailable)

Install. From the project root, ideally inside a virtual environment:

```
# from the project root; a virtual environment is recommended
python -m venv .venv
.venv\Scripts\activate          # Windows (use: source .venv/bin/activate on macOS/Linux)
pip install numpy pandas scikit-learn matplotlib nltk
```

Run the model. The script builds the “none” class, merges, trains the full feature-subset $\times$ depth grid, selects on the validation split, retrain the winner, and writes `results.md` into `outputs/`.

```
cd src/neural_network/scripts
python neural_network_code.py    # -> ../outputs/results.md
```

Regenerate the report figures. This reproduces the validation-selection heatmap, the depth-effect plot, and the confusion matrix into `src/neural_network/outputs/`. The gradient-health diagnostic (Figure 6, Table 2) is a separate script.

```
cd src/neural_network/scripts
python make_report_figures.py    # validation selection, depth, confusion
python gradient_diagnostics.py   # per-layer gradient health -> gradient_health.png
```

Compile this report. Run `pdflatex` from the `docs/` folder (twice, so cross-references and the table of figures resolve):

```
cd docs
pdflatex neural_network_report.tex # run twice to resolve cross-references
```